

Sistema FitLife Academia

Alunos:

Vitor Bitencourt

Arthur Brito Carvalho

Isael Canudo

Ana Clara

Gabriel Kawã

1. Introdução

O projeto **FitLife Academia** consiste em um sistema de gestão para academias desenvolvido na linguagem de programação Java, utilizando a biblioteca Swing para a interface gráfica de usuário (GUI). O principal objetivo do sistema é facilitar o cadastro e o gerenciamento de alunos e professores, a inscrição em modalidades, o registro de presença, e, principalmente, a gestão financeira e o acompanhamento de pagamentos dos alunos. A arquitetura do sistema foi concebida seguindo os princípios de Orientação a Objetos (POO), visando modularidade, reuso e fácil manutenção.

O sistema é dividido em pacotes lógicos como **Cadastro** (para Alunos e Professores), **Planos**, **Modalidades**, **Financeiro** (para Faturas e Pagamentos), e **Menu** (para a interface gráfica).

2. Funcionalidades Implementadas

As funcionalidades principais implementadas e testadas no sistema são:

2.1. Criação do Menu com Swing e Navegação

A interface do sistema foi construída integralmente com a biblioteca **Java Swing**, proporcionando uma experiência gráfica para o usuário.

- **FitLifeApp.java**: Atua como o contêiner principal (JFrame) e utiliza um **CardLayout** para gerenciar diferentes telas (painéis). Isso permite a transição suave entre a tela de Login, o formulário de Cadastro de Aluno/Professor, a seleção de Planos, e os *Dashboards* (Aluno, Professor e Admin).
- **Formulários de Cadastro**: Implementados em **AlunoFormPanel.java** e **ProfessorFormPanel.java** utilizando **GridBagLayout** para um design responsivo e organizado.

2.2. Verificação e Validação de Login

O sistema possui uma tela de login (`LoginPanel.java`) capaz de autenticar três perfis de usuário:

- **Aluno:** A autenticação é feita buscando o Aluno pelo CPF e validando a senha.
- **Professor:** Similar ao aluno, busca no cadastro de professores e valida a senha (embora a senha do professor não esteja explícita no construtor de `Professor.java`, a lógica de login tenta autenticá-lo).
- **Administrador:** Utiliza credenciais codificadas no código ("academia" / "123") para acesso ao `AdminDashboard`.

Além disso, o formulário de cadastro de professor inclui uma validação de CPF para garantir que apenas 11 dígitos sejam informados, e também uma validação de formato de horário (HH:mm) usando a classe auxiliar `TimeUtils`.

2.3. Aba de Pagamentos e Verificação de Status

A gestão financeira é centralizada nas classes do pacote `Financeiro`.

- **Modelo de Fatura:** A classe `Fatura` armazena dados essenciais como o aluno, o valor do plano, a data de vencimento e o **Status de Pagamento** (utilizando o `enum StatusPagamento`: PENDENTE, PAGO, ATRASADO).
- **Gestor Financeiro:** A classe `GestorFinanceiro` (implementada como **Singleton**) é responsável por `gerarNovaFatura` (automaticamente ao selecionar o plano ou manualmente pelo Admin) e `registrarPagamento`.
- **Verificação de Atraso:** O método `verificarAtraso()` em `Fatura` atualiza o status para `ATRASADO` se a data de vencimento for ultrapassada e o status ainda for `PENDENTE`.
- **Interface de Pagamento:** O `PagamentoPanel.java` exibe todas as faturas (ou apenas as do aluno logado) em uma `JList` com um `renderer` personalizado que usa ícones coloridos para indicar visualmente o status (Verde para PAGO, Laranja para PENDENTE e Vermelho para ATRASADO).

3. Conceitos de Programação Orientada a Objetos (POO)

A arquitetura do FitLife Academia é fortemente baseada em pilares da Programação Orientada a Objetos, o que contribui para a organização, flexibilidade e escalabilidade do código.

3.1. Herança (Inheritance)

A Herança foi amplamente utilizada para estabelecer hierarquias lógicas entre as entidades do sistema, permitindo que classes especializadas reutilizem a estrutura de classes mais genéricas.

Classe Pai (Genérica)	Classes Filhas (Especializadas)	Motivo da Utilização
Pessoa (Classe Abstrata)	Aluno, Professor	Reutilização dos atributos comuns (nome, CPF, idade) e implementação da regra de negócio específica (como tipo de usuário).
Aluno	AlunoVip	AlunoVip é uma especialização que herda todas as características de um Aluno comum, mas define um comportamento ou status específico (como o método isVip() e a validação do plano).
Plano	Mensal, Anual, VIP	Cada plano define seus próprios valores e duração (em dias), mas todos compartilham a estrutura básica de um Plano.
Modalidade (Classe Abstrata)	Yoga, Musculacao, Pilates	Permite tratar genericamente qualquer tipo de modalidade de exercício, enquanto a subclasse define o tipo específico.

3.2. Abstração e Encapsulamento

- **Abstração:** A classe abstrata `Pessoa` define o comportamento essencial de um indivíduo no sistema (possuir nome, CPF, idade e um tipo) sem especificar como esse indivíduo interage (isso é delegado às subclasses `Aluno` e `Professor`).
- **Encapsulamento:** Os atributos críticos das classes (como `cpf` em `Pessoa`, `senha` em `Aluno`, e `valor` em `Plano`) são declarados como `private` ou `protected`. O acesso e a modificação são controlados por métodos públicos (getters e setters), como o método `setPlano(Plano plano)` em `Aluno`, garantindo que a integridade dos dados seja mantida.

3.3. Polimorfismo (Polymorphism)

O Polimorfismo é evidente na sobrescrita do método `getTipo()`:

- `Pessoa` define o contrato (método abstrato `getTipo()`).
- `Aluno` e `Professor` implementam esse método de forma diferente, retornando "ALUNO", "PROFESSOR", ou "ALUNO VIP" (se o plano for VIP). Isso permite tratar um objeto como `Pessoa` genericamente, enquanto o comportamento específico de obter o tipo é determinado em tempo de execução.

Outros exemplos de Polimorfismo:

- O método `toString()` sobrescrito em `Fatura` e `RegistroTreino` para fornecer uma representação textual formatada.

• 3.4. Padrão Singleton

O padrão de projeto **Singleton** foi aplicado a duas classes essenciais, garantindo que haja apenas uma instância gerenciando os dados em todo o sistema, o que é crucial para consistência e centralização.

- **CadastroAcademico:** Garante um único ponto de acesso para as listas de todos os Alunos e Professores.
- **GestorFinanceiro:** Garante um único ponto de acesso para todas as operações e registros de `Fatura`.

4. Bibliotecas Utilizadas

O projeto FitLife Academia foi desenvolvido primariamente utilizando a API padrão do Java (Java SE), complementada com as bibliotecas essenciais para a interface gráfica e manipulação de estruturas de dados e datas.

Biblioteca/Pacote Principal	Pacotes/Classes Utilizadas	Propósito no Projeto
Java Swing	<code>javax.swing.*</code> (JFrame, JPanel, JLabel, JButton, JOptionPane, JList, etc.)	Construção da Interface Gráfica de Usuário (GUI). Elementos visuais como botões, campos de texto, <i>listas</i> e janelas de diálogo.
Java AWT	<code>java.awt.*</code> (Font, Color, BorderLayout, FlowLayout, GridBagLayout, Dimension, Component)	Gerenciamento de Layouts (como <code>GridBagLayout</code> e <code>CardLayout</code>) e recursos básicos de desenho (cores, fontes).
Java Util	<code>java.util.ArrayList</code> , <code>java.util.List</code> , <code>java.util.HashMap</code> , <code>java.util.Map</code>	Estruturas de Dados para armazenamento dinâmico de objetos (listas de alunos, professores, modalidades, faturas, e mapas de horários).
Java Time	<code>java.time.LocalDate</code>	Manipulação de datas para registro de emissão, pagamento e, crucialmente, cálculo de vencimento de faturas e registro de treinos.

5. Funcionalidades Não Concluídas

Durante o desenvolvimento, algumas funcionalidades previstas para o escopo completo não puderam ser finalizadas e integradas à versão atual.

5.1. Método de Reserva de Equipamentos

O método para a reserva de equipamentos (como esteiras, bicicletas ergométricas, etc.) foi concebido, mas **não implementado**.

- **Status Atual:** Não existe nenhuma classe ou método no código que manipule logicamente a disponibilidade ou a reserva de equipamentos por parte do aluno ou de outro usuário. A funcionalidade permanece como uma ideia na fase de planejamento.
- **Justificativa para a Não Conclusão:** O foco inicial do desenvolvimento foi direcionado para a implementação do núcleo do sistema (Cadastro, Login, Modalidades) e, principalmente, para a complexidade da **Gestão Financeira**, que envolveu a criação de um modelo de **Fatura**, um **GestorFinanceiro** (Singleton) e a lógica de verificação de atraso e exibição do status na GUI de pagamentos. O tempo de desenvolvimento foi prioritariamente alocado para garantir a estabilidade e a correção dos pilares de autenticação e pagamento, adiando a implementação da reserva de recursos físicos.

5.2. Aulas Especiais para os Clientes VIP

O projeto inclui um plano **VIP** e a classe **AlunoVip** (que verifica se o plano é VIP). No entanto, **não foi possível implementar a lógica para aulas especiais dedicadas a este grupo**.

- **Status Atual:** Um aluno pode ser identificado como VIP, mas o sistema de agendamento de aulas (**MenuProfessor.java** e **StudentDashboardPanel.java**) não distingue entre aulas regulares e aulas exclusivas para VIPs.
- **Justificativa para a Não Conclusão:** A implementação exigiria uma mudança na estrutura de agendamento, provavelmente a criação de uma nova classe ou atributo na classe **Modalidade** (ou uma especialização de **Modalidade**) para sinalizar o caráter VIP, bem como uma nova lógica de agendamento no **Dashboard** do Professor para vincular horários específicos apenas a alunos VIP. Essa complexidade adicional e a necessidade de reestruturação do agendamento levaram à sua suspensão temporária.

6. Dificuldades Enfrentadas

O desenvolvimento de um sistema coeso com interface gráfica envolveu desafios técnicos e de arquitetura.

- **Gestão de Estado e Múltiplos Painéis (Swing):** A principal dificuldade na camada de *front-end* foi a coordenação da navegação entre os diversos painéis (**JPanel**) utilizando o **CardLayout**. Foi necessário garantir que, ao trocar de tela, o estado do

usuário (e.g., `currentAluno`) fosse passado corretamente e que os dados exibidos nos *Dashboards* fossem atualizados (método `refresh()`), o que demandou a passagem da instância de `FitLifeApp` para a maioria dos painéis.

- **Persistência de Dados:** O sistema atual não possui persistência em banco de dados ou arquivo. Isso significa que, a cada reinicialização da aplicação, todos os dados de cadastro e faturas são perdidos. A ausência de um sistema de persistência, embora comum em protótipos, foi um obstáculo para testar cenários de longo prazo (como o cálculo de atraso de faturas em datas futuras).
- **Acoplamento do Login/Regras:** Algumas regras de negócio, como as credenciais de *Admin* ("academia" / "123") ou a lógica de validação de CPF e Horário, estão diretamente nos painéis da GUI, o que dificulta a reutilização e a alteração dessas regras (alto acoplamento).

7. Referências Consultadas

Durante o desenvolvimento do projeto, foram consultados diversos recursos para implementação e para a aplicação correta dos conceitos de Java e POO.

- **Documentação Java SE:** Para a utilização correta das classes e APIs padrão, como os pacotes `java.util` (`List`, `ArrayList`, `HashMap`) e `java.time` (`LocalDate`).
- **Tutoriais de Java Swing:** Para a implementação da interface gráfica (layout managers e componentes avançados como `JList` com `CellRenderer`).
- **Materiais sobre Padrões de Projeto:** Para a aplicação do padrão **Singleton** nas classes `CadastroAcademico` e `GestorFinanceiro`

8. Conclusão

O sistema FitLife Academia atende aos requisitos essenciais de um sistema de gestão de academia para um protótipo inicial. As funcionalidades de cadastro, login com validação de perfis, e a crucial gestão financeira com controle de faturas e status de pagamento foram implementadas com sucesso, utilizando os princípios da Programação Orientada a Objetos para criar um código modular e robusto.

Apesar dos desafios relacionados à interface gráfica e o acoplamento de regras, a estrutura do projeto demonstra uma clara separação de responsabilidades em pacotes (`Cadastro`, `Financeiro`, `Menu`) e um uso consistente de conceitos como Herança (`Pessoa` -> `Aluno/Professor`) e o padrão Singleton.

Para futuras iterações, o foco deve ser na implementação das funcionalidades pendentes (reserva de equipamentos e aulas VIP) e, principalmente, na adição de uma camada de persistência para garantir que os dados sejam salvos permanentemente, migrando as regras de negócio para fora da camada de apresentação (GUI).